INSTITUTO NACIONAL DE APRENDIZAJE

Núcleo Comercio y Servicios · Subsector Informática y Comunicación

Diseño de algoritmos

Sesión 19 · Control de flujo: ciclos y switch

UA2 · Programación estructurada | Módulo CSTI12010



Persona facilitadora

Giovanni Antonio Coto Calderón

De PSeInt a C: estructuras de control

PSeInt	C	¿Cuándo se usa?
`Mientras c Hacer … FinMientras`	`while (c) { … }`	Puede no repetir nunca
`Repetir … Hasta Que c`	`do { … } while (!c);`	Al menos una vez
`Para i <- 1 Hasta n Con Paso 1`	`for (i = 1; i <= n; i++)`	Se sabe cuántas veces
`Segun x Hacer  1: …`	`switch (x) { case 1: … break; }`	Un valor, varias opciones
`De Otro Modo:`	`default:`	Ninguna coincide

Ojo con **Repetir**: en C la condición del **do … while** dice cuándo **seguir**, no cuándo **terminar**. Se escribe la condición **contraria**.

while: preguntar, repetir, actualizar

Todo ciclo **while** necesita tres pasos: **inicializar** la variable, **preguntar** en la condición y **actualizarla** dentro del cuerpo.

while.c

```c
#include <stdio.h>
int main(void) {
    int i = 1;                  // 1. inicializar
    while (i <= 5) {            // 2. preguntar
        printf("Vuelta numero %d\n", i);
        i++;                    // 3. actualizar
    }
    printf("Fin del ciclo\n");
    return 0;
}
```

Salida real (gcc)

```
Vuelta numero 1
Vuelta numero 2
Vuelta numero 3
Vuelta numero 4
Vuelta numero 5
Fin del ciclo
```

for: un ciclo de conteo en una línea

for (inicio; condición; actualización) reúne los tres pasos del while. Es el **Para** de PSeInt: **Para i <- 5 Hasta 1 Con Paso -1** → **for (i = 5; i >= 1; i--)**.

regresiva.c

```c
#include <stdio.h>
int main(void) {
    int i;
    // inicio; condicion; actualizacion
    for (i = 5; i >= 1; i--) {
        printf("%d\n", i);
    }
    printf("Despegue\n");
    return 0;
}
```

Salida real (gcc)

```
5
4
3
2
1
Despegue
```

do … while: validar datos

El cuerpo se ejecuta **primero** y luego se pregunta. Ideal para **validar**: se pide el dato hasta que sea correcto. Note el **;** después del while.

nota_valida.c (fragmento)

```c
do {
    printf("Nota (0 a 100): ");
    scanf("%lf", &nota);
    if (nota < 0 || nota > 100) {
        printf("Nota invalida\n");
    }
} while (nota < 0 || nota > 100);
printf("Nota registrada: %.1f\n", nota);
```

Salida real (105 y 85)

```
Nota (0 a 100): 105
Nota invalida
Nota (0 a 100): 85
Nota registrada: 85.0
```

Centinela: AhorroMensual (sesión 7)

Un **centinela** (-1) marca el fin de los datos. Se lee el primer dato **antes** del while y el siguiente **al final** del cuerpo.

ahorro.c (fragmento)

```c
int meses = 0;
double deposito;
double totalAhorrado = 0;
printf("Deposito del mes (-1 para terminar): ");
scanf("%lf", &deposito);
while (deposito != CENTINELA) {
    meses++;
    totalAhorrado += deposito;
    printf("Deposito del mes (-1 para terminar): ");
    scanf("%lf", &deposito);
}
printf("Meses con deposito: %d\n", meses);
printf("Total ahorrado: %.2f\n", totalAhorrado);
```

Salida real

```
Deposito del mes (-1 para terminar): 5000
Deposito del mes (-1 para terminar): 3000
Deposito del mes (-1 para terminar): 7000
Deposito del mes (-1 para terminar): -1
Meses con deposito: 3
Total ahorrado: 15000.00
```

for anidados: filas y columnas

El ciclo **interno** completa **todas** sus vueltas por **cada** vuelta del externo: 3 filas × 5 columnas = **15** productos.

anidados.c

```c
#include <stdio.h>
int main(void) {
    int fila, col;
    for (fila = 1; fila <= 3; fila++) {      // externo
        for (col = 1; col <= 5; col++) {     // interno
            printf("%4d", fila * col);
        }
        printf("\n");     // fin de la fila
    }
    return 0;
}
```

Salida real (gcc)

```
   1   2   3   4   5
   2   4   6   8  10
   3   6   9  12  15
```

SABER 3

break y continue

continue salta al **siguiente** paso del ciclo. **break termina** el ciclo. Se usan dentro de un if.

saltos.c (sin #include)

```c
int main(void) {
    int i;
    for (i = 1; i <= 10; i++) {
        if (i % 3 == 0) {
            continue;     // salta al siguiente i
        }
        if (i > 7) {
            break;        // termina el ciclo
        }
        printf("%d ", i);
    }
    printf("\nFin con i = %d\n", i);
    return 0;
}
```

Salida real (gcc)

```
1 2 4 5 7
Fin con i = 8
```

exit(): terminar todo el programa

exit(1) (de **stdlib.h**) termina el programa **completo** desde cualquier lugar. El número indica al sistema si hubo error (distinto de 0).

salir.c

```c
#include <stdio.h>
#include <stdlib.h>        // exit()
int main(void) {
    int cantidad;
    printf("Cantidad de notas: ");
    scanf("%d", &cantidad);
    if (cantidad <= 0) {
        printf("Cantidad invalida\n");
        exit(1);           // termina TODO el programa
    }
    printf("Se leeran %d notas\n", cantidad);
    return 0;
}
```

Salida real (cantidad 0)

```
Cantidad de notas: 0
Cantidad invalida
(código de salida: 1)
```

SABER 4

switch: ConversorMoneda (sesión 11)

switch compara un valor **entero** (o char) con cada **case**. **break** cierra cada caso; **default** es el De Otro Modo.

conversor.c (fragmento)

```
switch (opcion) {
    case 1:
        printf("Colones: ");
        scanf("%lf", &monto);
        printf("Dolares: %.2f\n", monto / 510);
        break;
    case 2:
        printf("Dolares: ");
        scanf("%lf", &monto);
        printf("Colones: %.2f\n", monto * 510);
        break;
    default:
        printf("Opcion invalida\n");
}
```

Salida real (2 y 100)

```
1. Colones a dolares  2. Dolares a colones
Opcion: 2
Dolares: 100
Colones: 51000.00
```

CUIDADO

switch sin break

Sin **break**, la ejecución **sigue** al case siguiente. gcc lo advierte con **-Wextra**: «this statement may fall through».

Incorrecto

Con opcion = 1 muestra las tres

```
case 1:
    printf("Colones a dolares\n");
case 2:
    printf("Dolares a colones\n");
default:
    printf("Opcion invalida\n");
// Con opcion = 1 se muestran
// las tres lineas
```

Correcto

Un break por case

```
case 1:
    printf("Colones a dolares\n");
    break;
case 2:
    printf("Dolares a colones\n");
    break;
default:
    printf("Opcion invalida\n");
```

goto: por qué se evita

goto salta a una etiqueta

Existe, pero no se usa en el curso

- inicio: printf("%d\n", i); i++;
- if (i <= 3) { goto inicio; }
- Imprime 1, 2 y 3, como un ciclo.
- Con muchos saltos: «código espagueti».

Programación estructurada

Una entrada y una salida

- for (i = 1; i <= 3; i++) { printf("%d\n", i); }
- Se lee de arriba hacia abajo.
- Una entrada y una salida por estructura.
- Todo goto se reemplaza por if, while o for.